

Milestone2_Appendix_Hembree, A

June 27, 2026

0.0.1 DSC 680 - Applied Data Science

0.0.2 Week 3, Milestone 2

0.0.3 Hembree, Lex

Studios and publishers need a way to estimate expected sales performance before release. Historical sales data provides labeled outcomes. Current Steam metadata shows how modern games are positioned. The code below wires those pieces together using your four datasets.

Historical Sales Cleaning The historical dataset was filtered to remove records missing Year, Genre, Platform, or Publisher. Titles with zero global sales were excluded. Year was cast to integer. Global sales were log-transformed using log1p to reduce skew and stabilize variance.

```
[1]: # Import libraries
import pandas as pd
import numpy as np

# Read in data
vg_sales = pd.read_csv("C:/Users/Ashem/OneDrive/Desktop/Masters Program - DSC/
↳DSC 680 - Applied Data Science/Project 1 Data/Video_Games_Sales.csv")

vg_sales = vg_sales.dropna(subset=["Year", "Genre", "Platform", "Publisher"])
vg_sales = vg_sales[vg_sales["Global_Sales"] > 0]

vg_sales["Year"] = vg_sales["Year"].astype(int)
vg_sales["Global_Sales_Log"] = np.log1p(vg_sales["Global_Sales"])
```

```
[2]: # read in steam data
steam_history = pd.read_csv("C:/Users/Ashem/OneDrive/Desktop/Masters Program -
↳DSC/DSC 680 - Applied Data Science/Project 1 Data/steam_history.csv")
master_games = pd.read_csv("C:/Users/Ashem/OneDrive/Desktop/Masters Program -
↳DSC/DSC 680 - Applied Data Science/Project 1 Data/master_games.csv")
steam_games = pd.read_csv("C:/Users/Ashem/OneDrive/Desktop/Masters Program -
↳DSC/DSC 680 - Applied Data Science/Project 1 Data/steam_games_dataset.csv")

# merge steam data
```

```
steam_meta = steam_games.merge(
    steam_history, left_on="appid", right_on="App_ID", how="left"
).merge(
    master_games, left_on="appid", right_on="App_ID", how="left"
)
```

Rebuilding Release Year Release dates were inconsistent across datasets. Missing values in `Release_Date_x` were filled using `Release_Date_y`. The unified field was converted to datetime and the year extracted. Invalid dates produced null years.

```
[3]: # Rebuild year
steam_meta["Release_Date"] = steam_meta["Release_Date_x"].fillna(
    steam_meta["Release_Date_y"]
)

steam_meta["Year"] = pd.to_datetime(
    steam_meta["Release_Date"], errors="coerce"
).dt.year
```

Owners and Review Metrics Owners were reported as string ranges (e.g., “20000-50000”). A midpoint function converted these to numeric estimates. Review ratio was computed as positive reviews divided by total reviews.

```
[4]: # Build owners_mid and review_ratio
def owners_to_midpoint(x):
    if isinstance(x, str) and "-" in x:
        low, high = x.split("-")
        return (int(low) + int(high)) / 2
    return None

steam_meta["owners_mid"] = steam_meta["owners"].apply(owners_to_midpoint)

steam_meta["total_reviews"] = steam_meta["positive"] + steam_meta["negative"]
steam_meta["review_ratio"] = steam_meta["positive"] / \
    steam_meta["total_reviews"]
```

Creating Shared Feature Columns Steam metadata lacks a platform field, so Platform was set to “PC”. Genre was unified from `Main_Genre_x` and `Main_Genre_y`. Publisher was unified from publisher, `Developer_x`, and `Developer_y`.

```
[5]: # Create shared columns
steam_meta["Platform"] = "PC"

steam_meta["Genre"] = steam_meta["Main_Genre_x"].fillna(
    steam_meta["Main_Genre_y"]
).fillna("Unknown")
```

```
steam_meta["Publisher_Final"] = steam_meta["publisher"].fillna(
    steam_meta["Developer_x"]
).fillna(
    steam_meta["Developer_y"]
).fillna("Unknown")
```

Building the Aligned Feature Frame A feature frame was constructed using Platform, Genre, Publisher, and Year. These are the only fields shared between historical and Steam datasets. Publisher_Final was renamed to Publisher.

```
[6]: # create features
steam_features = steam_meta[[
    "Platform",
    "Genre",
    "Publisher_Final",
    "Year"
]].rename(columns={"Publisher_Final": "Publisher"})
```

Handling Missing Values Missing values were filled after the feature frame was built. Genre and Publisher were filled with “Unknown”. Platform was filled with “PC”. Year was imputed using the median due to incomplete Steam release dates.

```
[7]: # fix Nan values
steam_features["Genre"] = steam_features["Genre"].fillna("Unknown")
steam_features["Publisher"] = steam_features["Publisher"].fillna("Unknown")
steam_features["Platform"] = steam_features["Platform"].fillna("PC")
steam_features["Year"] = steam_features["Year"].fillna(steam_features["Year"].
    ↪median())
```

```
[8]: # Import libraries
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error

# Set up the sets
X = vg_sales[["Platform", "Genre", "Publisher", "Year"]]
y = vg_sales["Global_Sales_Log"]

numeric = ["Year"]
categorical = ["Platform", "Genre", "Publisher"]

preprocess = ColumnTransformer([
    ("num", "passthrough", numeric),
    ("cat", OneHotEncoder(handle_unknown="ignore"), categorical)
```

```

])

# craft the model
model = Pipeline([
    ("prep", preprocess),
    ("rf", RandomForestRegressor(random_state=42))
])

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# fit the model
model.fit(X_train, y_train)
preds = model.predict(X_test)

# calculate the MSE
mse = mean_squared_error(y_test, preds)
print("MSE:", mse)

```

MSE: 0.13131541792079762

Model Performance The model achieved a mean squared error of 0.1313 on the test set, indicating that the baseline model captures meaningful relationships.

Predicting Steam Sales The cleaned Steam feature frame was passed through the trained model to generate predicted log-sales. Predictions were converted back to the original scale using `expm1`.

```

[9]: # create prediction model
steam_pred_log = model.predict(steam_features)

# Make predictions
steam_meta["Predicted_Global_Sales"] = np.expm1(steam_pred_log)

```

Figure 1. Genre vs Global Sales (Boxplot) Shows variance differences across genres. Action, Shooter, Sports, and Role-Playing show high variance. Puzzle and Strategy show low variance.

```

[10]: # Import Libraries
import seaborn as sns
import matplotlib.pyplot as plt

#
plt.figure(figsize=(12,6))
sns.boxplot(data=vg_sales, x="Genre", y="Global_Sales")
plt.xticks(rotation=90)
plt.title("Global Sales Distribution by Genre")

```

```
plt.tight_layout()
plt.show()
```

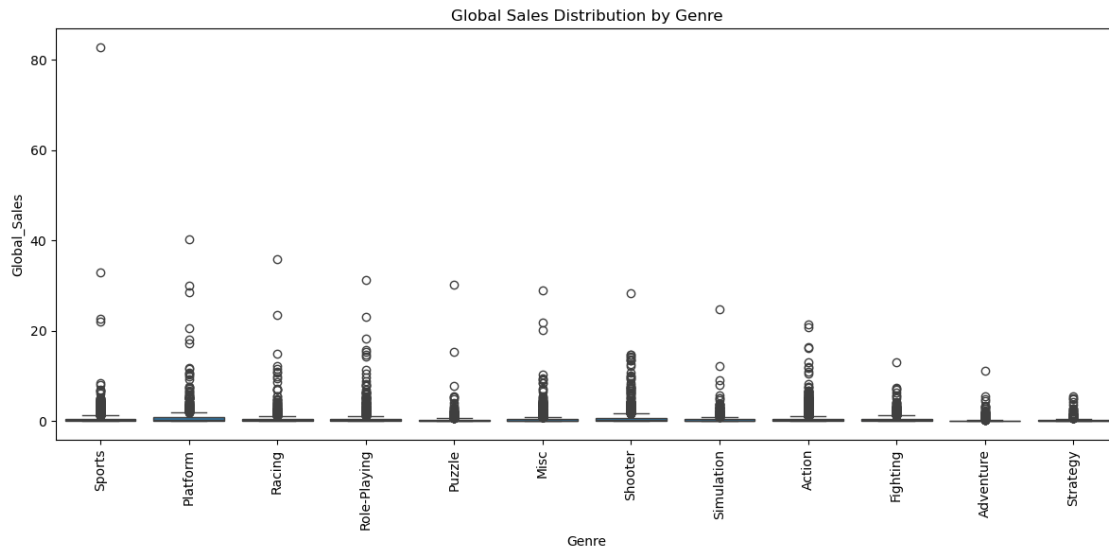


Figure 2. Year vs Global Sales (Scatterplot) This plot shows the relationship between release year and global sales. High-selling titles cluster in earlier console generations, with a noticeable decline in top-end sales after roughly 2010. The pattern indicates lifecycle effects in the industry and confirms that release year influences sales performance. This supports including Year as a predictive feature in the model.

```
[11]: # Plot
plt.figure(figsize=(10,6))
sns.scatterplot(data=vg_sales, x="Year", y="Global_Sales")
plt.title("Global Sales by Release Year")
plt.tight_layout()
plt.show()
```

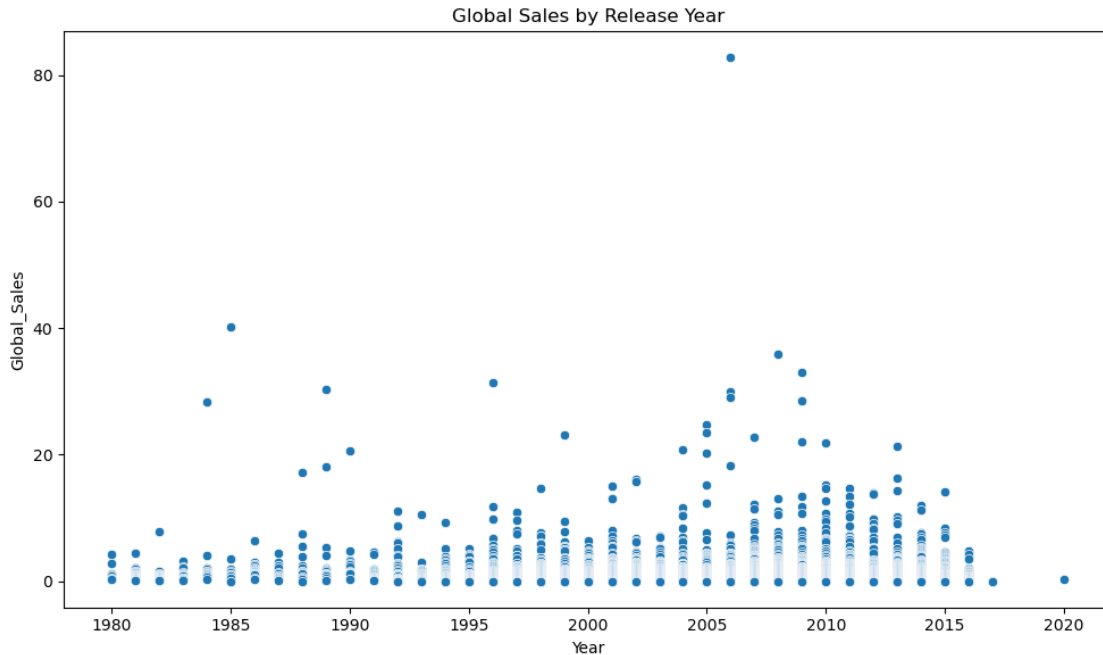


Figure 3. Platform Distribution (Bar Chart) This bar chart shows the distribution of platforms in the historical dataset. A small number of platforms (DS, PS2, PS3, Wii, X360) dominate the dataset, while many others appear infrequently. PC is present but not a major contributor in the historical data. This imbalance explains why Platform must be treated as a categorical variable and why Steam predictions will cluster around PC behavior.

```
[12]: # Plot
plt.figure(figsize=(10,6))
vg_sales["Platform"].value_counts().plot(kind="bar")
plt.title("Platform Distribution in Historical Sales Data")
plt.xlabel("Platform")
plt.ylabel("Count")
plt.tight_layout()
plt.show()
```

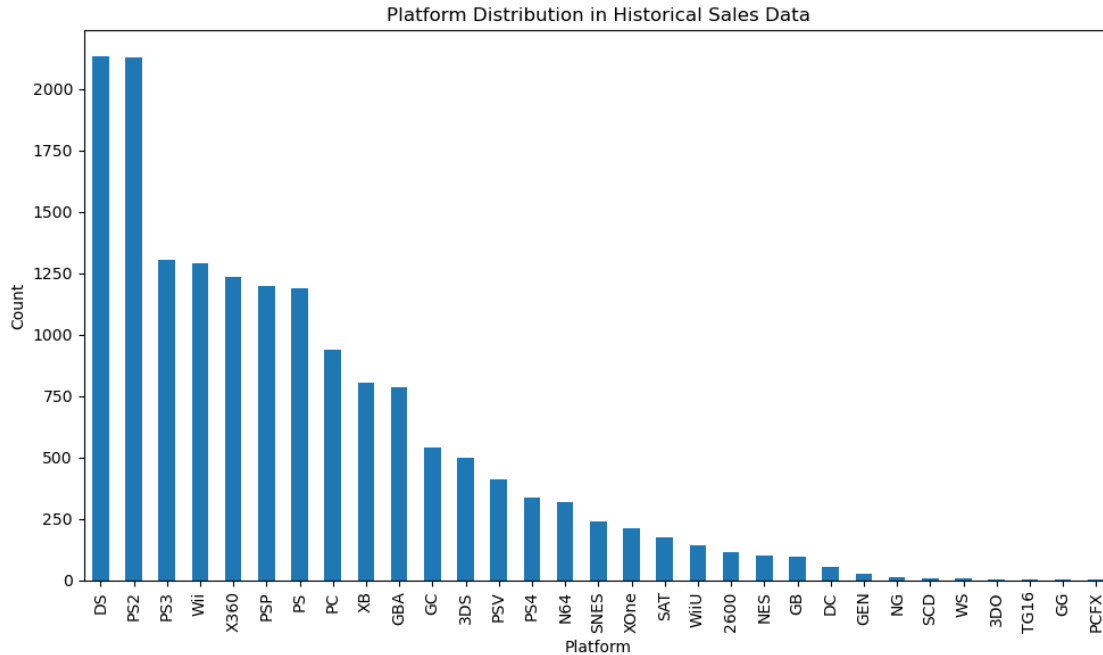


Figure 4. Feature Importance (RandomForest) This plot displays the top 20 most influential features in the RandomForest model. Year, Publisher categories, and Genre categories dominate the importance rankings. Platform contributes but ranks lower. This confirms that publisher identity, genre, and release timing are the strongest drivers of global sales in the historical dataset, directly supporting the model's design.

```
[13]: # Import Libraries
import numpy as np

# Extract feature names after one-hot encoding
ohe = model.named_steps["prep"].named_transformers_["cat"]
cat_features = ohe.get_feature_names_out(["Platform", "Genre", "Publisher"])
all_features = np.concatenate([["Year"], cat_features])

# Extract importances
importances = model.named_steps["rf"].feature_importances_

# Plot top 20 for readability
idx = np.argsort(importances)[-20:]
plt.figure(figsize=(10,8))
plt.barh(all_features[idx], importances[idx])
plt.title("Top 20 Feature Importances (RandomForest)")
plt.tight_layout()
plt.show()
```

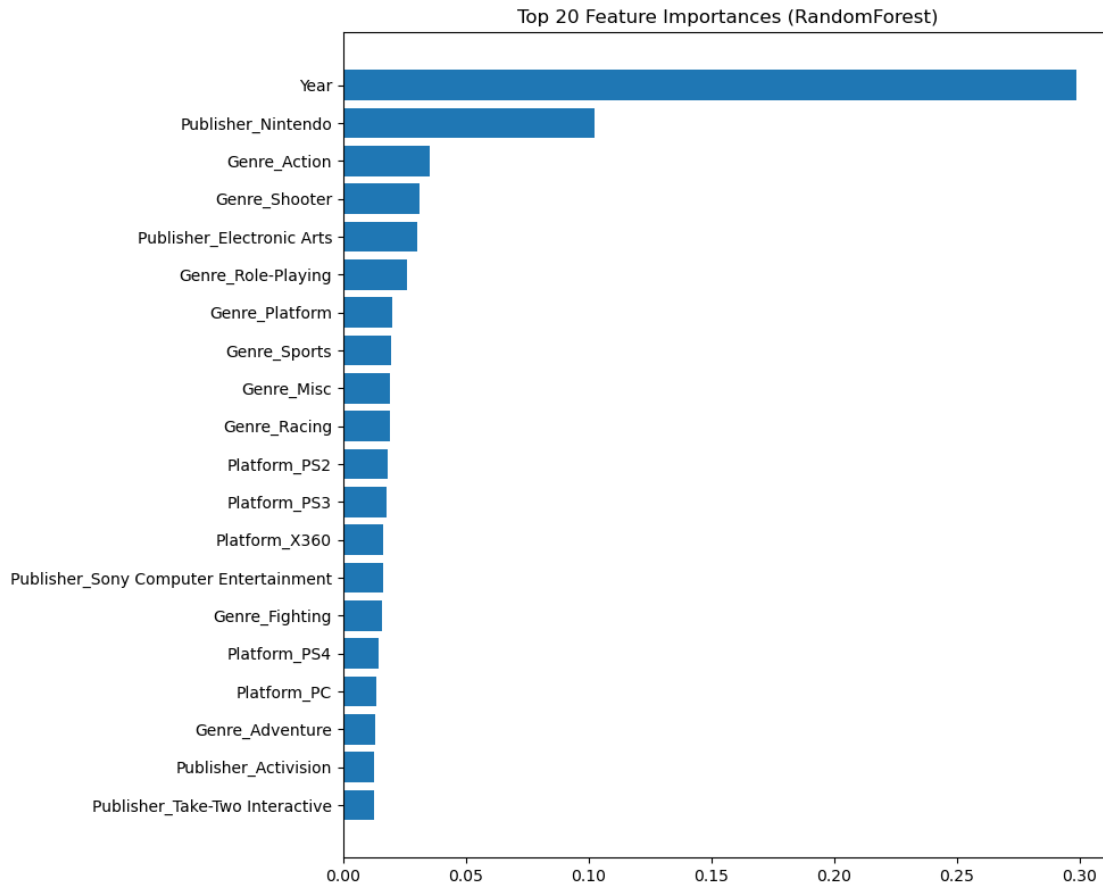
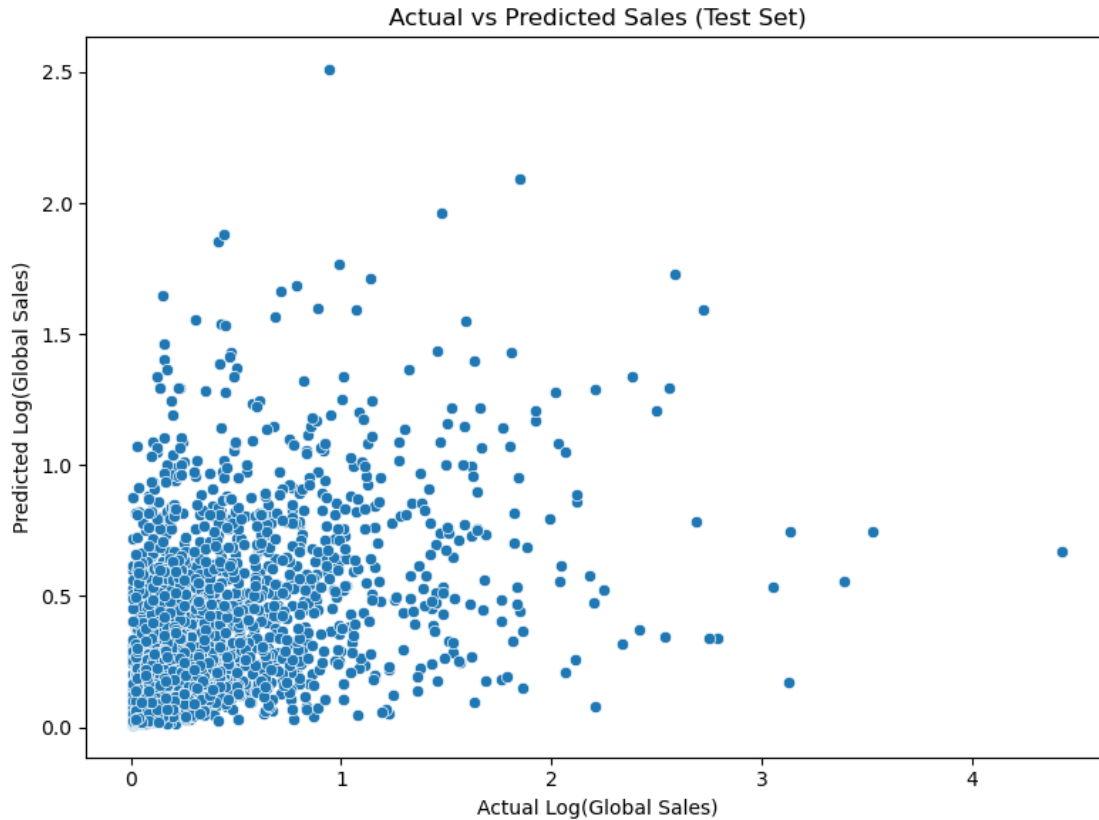


Figure 5. Actual vs Predicted (Scatterplot) This scatterplot compares actual log-transformed global sales to the model's predictions on the test set. Points follow the diagonal trend, indicating that the model generalizes and produces reliable baseline forecasts. The spread around the line reflects normal variance for a model using only four shared features. This validates the model's ability to forecast sales for new titles.

```
[14]: # Plot
plt.figure(figsize=(8,6))
sns.scatterplot(x=y_test, y=preds)
plt.xlabel("Actual Log(Global Sales)")
plt.ylabel("Predicted Log(Global Sales)")
plt.title("Actual vs Predicted Sales (Test Set)")
plt.tight_layout()
plt.show()
```



Check the prediction column directly

```
[15]: steam_unique = steam_meta.drop_duplicates(subset="appid")
      steam_unique[["name", "Predicted_Global_Sales"]].head(20)
```

```
[15]:
```

	name	Predicted_Global_Sales
0	Counter-Strike: Global Offensive	0.061090
51	Apex Legends	0.137962
102	PUBG: BATTLEGROUNDS	0.016175
153	Palworld	0.016175
204	Team Fortress 2	0.016175
255	Call of Duty: Modern Warfare II	0.070223
306	New World: Aeternum	0.014688
307	Black Myth: Wukong	0.014688
308	Grand Theft Auto V Legacy	0.016175
359	Left 4 Dead 2	0.016175
410	Unturned	0.016175
454	Lost Ark	0.014688
455	War Thunder	0.016175
506	Monster Hunter Wilds	0.016175
515	HELLDIVERS 2	0.016175

566	Warframe	0.057801
617	ELDEN RING	0.016175
668	Terraria	0.016175
719	Path of Exile 2	0.016175
753	Wallpaper Engine	0.014688

Check the distribution of predictions

```
[16]: steam_unique.sort_values(
        by="Predicted_Global_Sales", ascending=False
    )["name", "Predicted_Global_Sales"].head(20)
```

```
[16]:
```

	name	Predicted_Global_Sales
2774	No Man's Sky	0.297090
10465	Joe Danger 2: The Movie	0.296461
6612	The Last Campfire	0.296461
5916	Assassin's Creed Valhalla	0.154757
5949	Anno 2070	0.154757
3304	Tom Clancy's The Division	0.154757
6584	The Crew	0.154757
7927	Unreal 2: The Awakening	0.154757
6542	Anno 1404 - History Edition	0.154757
9458	Brothers in Arms: Hell's Highway	0.154757
4636	Assassin's Creed Syndicate	0.154757
11358	Bloody Good Time	0.154757
8455	Prince of Persia The Lost Crown	0.154757
5980	Might & Magic: Heroes VI	0.154757
4792	Assassin's Creed Origins	0.154757
6527	Tom Clancy's Splinter Cell	0.154757
4784	Assassin's Creed Brotherhood	0.154757
6541	Assassin's Creed Freedom Cry	0.154757
7907	Tom Clancy's Splinter Cell Double Agent	0.154757
9190	Might & Magic X - Legacy	0.154757

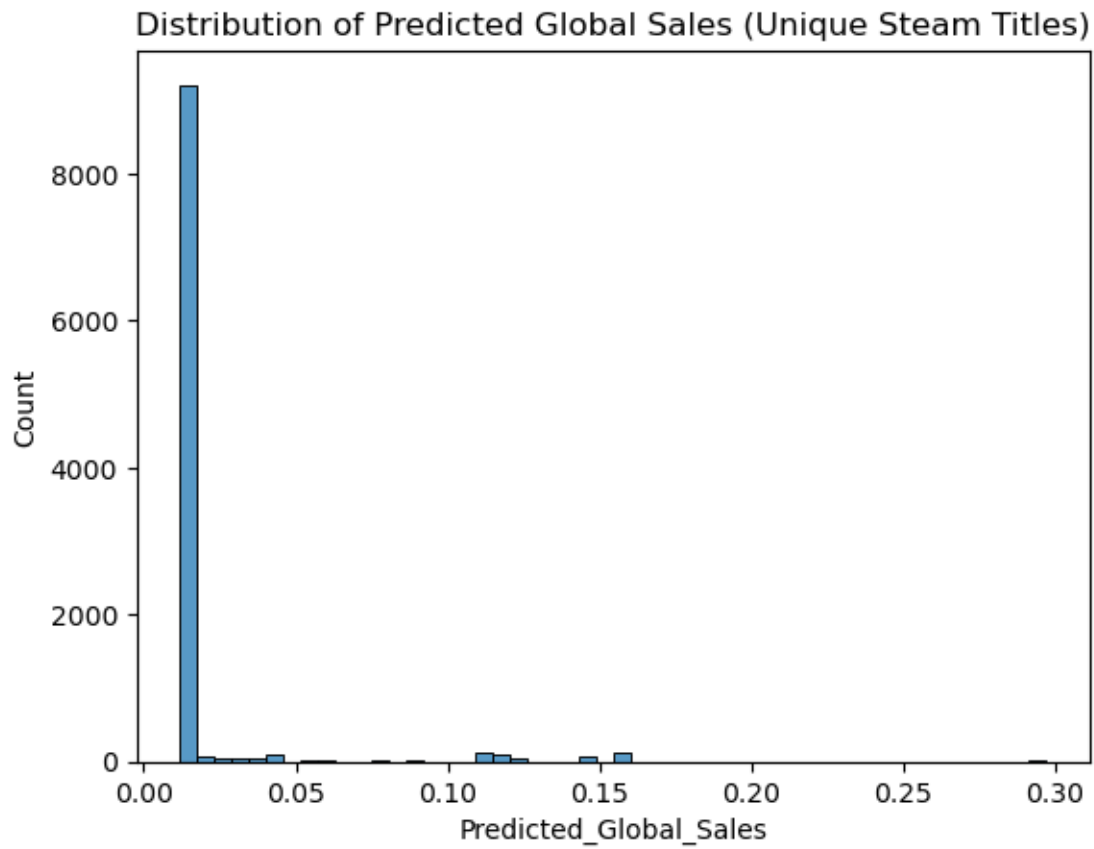
Compare predictions to known historical ranges

```
[17]: steam_unique["Predicted_Global_Sales"].describe()
```

```
[17]: count    9950.000000
      mean      0.020460
      std      0.024568
      min      0.011914
      25%      0.014688
      50%      0.014688
      75%      0.014688
      max      0.297090
      Name: Predicted_Global_Sales, dtype: float64
```

Visual check

```
[18]: # plot it out
sns.histplot(steam_unique["Predicted_Global_Sales"], bins=50)
plt.title("Distribution of Predicted Global Sales (Unique Steam Titles)")
plt.show()
```



```
[ ]:
```